

# PERTEMUAN 3 - STRUKTUR DATA, ADT, DAN ANALISIS KOMPLEKSITAS

- Nama: Nadine Valia Azzahra
- NIM: 2C2230007
- PRODI: Sains Data

## 1. JUDUL PRAKTIKUM

KONSEP STRUKTUR DATA, ADT, DAN ANALISIS KOMPLEKSITAS ALGORITMA

## 2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep dasar struktur data dalam pemrograman.
2. Menjelaskan fungsi dan manfaat struktur data dalam pengelolaan data.
3. Memahami konsep Abstract Data Type (ADT).
4. Mengimplementasikan ADT sederhana menggunakan Python.
5. Memahami pentingnya efisiensi algoritma dalam menyelesaikan permasalahan komputasi.
6. Mengidentifikasi kompleksitas waktu suatu algoritma.
7. Menggunakan Big-O Notation untuk menganalisis performa algoritma sederhana.
8. Membandingkan efisiensi beberapa algoritma berdasarkan jumlah proses yang dilakukan.

## 3. DASAR TEORI

### Struktur Data

Struktur data adalah cara menyimpan dan mengorganisasikan data di dalam komputer agar dapat diakses dan dikelola secara efisien. Pemilihan struktur data yang tepat dapat meningkatkan kecepatan pemrosesan data serta penggunaan memori. Contoh struktur data yang sering digunakan adalah list, stack, queue, tree, dan graph.

### Abstract Data Type

Abstract Data Type (ADT) adalah model konseptual yang mendefinisikan jenis data dan operasi yang dapat dilakukan tanpa memperhatikan detail implementasinya. Salah satu contoh ADT adalah stack yang menggunakan prinsip Last In First Out (LIFO) dengan operasi utama push dan pop.

### Efisiensi Algoritma

Efisiensi algoritma menunjukkan seberapa baik suatu algoritma bekerja dalam menyelesaikan masalah. Algoritma yang efisien dapat memproses data lebih cepat dan menggunakan sumber daya komputer secara optimal, terutama ketika jumlah data yang diolah semakin besar.

### Kompleksitas Algoritma dan Big-O Notation

Kompleksitas algoritma digunakan untuk mengukur kinerja algoritma berdasarkan waktu eksekusi dan penggunaan memori. Big-O Notation merupakan metode yang digunakan untuk menggambarkan pertumbuhan kompleksitas algoritma terhadap ukuran data masukan. Beberapa kompleksitas yang umum digunakan adalah  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ , dan  $O(n^2)$ .

### Perbandingan Efisiensi Algoritma

Setiap algoritma memiliki tingkat efisiensi yang berbeda. Sebagai contoh, Linear Search melakukan pencarian data satu per satu, sedangkan Binary Search membagi data menjadi beberapa bagian sehingga proses pencarian dapat dilakukan lebih cepat pada data yang terurut. Oleh karena itu, pemilihan algoritma yang tepat sangat penting untuk meningkatkan performa program.

## ✓ TUGAS PRAKTIKUM

### ✓ Studi Kasus 1-4

```
# STUDI KASUS 1: Penyimpanan Data Mahasiswa

# Membuat list kosong untuk menyimpan nama mahasiswa
mahasiswa = []

# Input jumlah mahasiswa
jumlah = int(input("Masukkan jumlah mahasiswa: "))

# Input nama mahasiswa satu per satu
for i in range(jumlah):
    nama = input(f"Masukkan nama mahasiswa ke-{i+1}: ")
    mahasiswa.append(nama)
```

```
# Menampilkan daftar mahasiswa
print("\nDaftar nama mahasiswa:")
for nama in mahasiswa:
    print(f" - {nama}")
```

```
Masukkan jumlah mahasiswa: 3
Masukkan nama mahasiswa ke-1: San
Masukkan nama mahasiswa ke-2: Mingi
Masukkan nama mahasiswa ke-3: Yunho
```

```
Daftar nama mahasiswa:
- San
- Mingi
- Yunho
```

## ✓ Analisis:

Menggunakan struktur data dynamic list ([]) untuk menampung elemen string secara berurutan. Menggunakan perulangan for berbasis range() untuk menambahkan data secara dinamis via .append().

```
# STUDI KASUS 2: Simulasi Stack Buku (LIFO)

# Membuat list kosong untuk stack buku
tumpukan_buku = []

# Input jumlah buku
jumlah = int(input("Masukkan jumlah buku: "))

# Menambahkan buku ke stack
for i in range(jumlah):
    buku = input(f"Masukkan nama buku ke-{i+1}: ")
    tumpukan_buku.append(buku)

# Menampilkan isi stack
print("\nBuku ditambahkan:", tumpukan_buku)

# Mengambil buku paling atas (LIFO)
buku_diambil = tumpukan_buku.pop()

print(f"Buku yang diambil (LIFO): {buku_diambil}")
print("Sisa tumpukan:", tumpukan_buku)
```

```
Masukkan jumlah buku: 3
Masukkan nama buku ke-1: Math
Masukkan nama buku ke-2: English
Masukkan nama buku ke-3: Economic
```

```
Buku ditambahkan: ['Math', 'English', 'Economic']
```

Buku yang diambil (LIFO): Economic  
Sisa tumpukan: ['Math', 'English']

## ✓ Analisis:

Mengimplementasikan konsep Abstract Data Type (ADT) Stack dengan prinsip Last In First Out (LIFO). Operasi `append()` bertindak sebagai push (menambah ke atas) dan `.pop()` otomatis mengambil elemen paling terakhir dimasukkan.

```
# STUDI KASUS 3: Analisis Perulangan (Single vs Nested Loop)

n = 4

# Single loop - O(n)
iterasi_single = 0
print(f"\nSingle Loop (n={n}) - Kompleksitas O(n):")
for i in range(n):
    iterasi_single += 1
    print(f" i = {i}")
print(f" Total iterasi: {iterasi_single}")

# Nested loop - O(n^2)
iterasi_nested = 0
print(f"\nNested Loop (n={n}) - Kompleksitas O(n^2):")
for i in range(n):
    for j in range(n):
        iterasi_nested += 1
        print(f" i={i}, j={j}")
print(f" Total iterasi: {iterasi_nested}")

print(f"\nKesimpulan: Single loop = {iterasi_single} iterasi, "
      f"Nested loop = {iterasi_nested} iterasi")
```

Single Loop (n=4) - Kompleksitas O(n):

```
i = 0
i = 1
i = 2
i = 3
Total iterasi: 4
```

Nested Loop (n=4) - Kompleksitas O(n^2):

```
i=0, j=0
i=0, j=1
i=0, j=2
i=0, j=3
i=1, j=0
i=1, j=1
i=1, j=2
i=1, j=3
```

```

i=2, j=0
i=2, j=1
i=2, j=2
i=2, j=3
i=3, j=0
i=3, j=1
i=3, j=2
i=3, j=3
Total iterasi: 16

```

Kesimpulan: Single loop = 4 iterasi, Nested loop = 16 iterasi

## ✓ Analisis:

Membuktikan perbedaan pertumbuhan Big-O Notation. Single loop berjalan secara linier dengan kompleksitas  $O(n)$ , sedangkan nested loop berjalan secara kuadratik dengan kompleksitas  $O(n^2)$ .

```

# STUDI KASUS 4: Pencarian Data - Linear Search

daftar_id = [1021, 1034, 1045, 1056, 1067, 1078, 1089]
target_id = 1056

print(f"Daftar ID anggota: {daftar_id}")
print(f"Mencari ID: {target_id}")

ditemukan = False
for id_anggota in daftar_id:
    if id_anggota == target_id:
        ditemukan = True
        break

if ditemukan:
    print(f" ID {target_id} ditemukan dalam daftar anggota koperasi.")
else:
    print(f" ID {target_id} tidak ditemukan.")

Daftar ID anggota: [1021, 1034, 1045, 1056, 1067, 1078, 1089]
Mencari ID: 1056
ID 1056 ditemukan dalam daftar anggota koperasi.

```

## Analisis:

Menerapkan pencarian sekuensial dengan memeriksa elemen satu per satu dari awal. Kata kunci break digunakan untuk menghentikan proses iterasi segera setelah target\_id ditemukan, mengoptimalkan waktu eksekusi.

## ✓ Tugas Level Mudah

```
# Tugas 1 - Menyimpan 5 angka dalam list
```

```
angka = [15, 30, 45, 60, 75]
print("Data angka dalam list:", angka)
print("Menampilkan satu per satu:")
for a in angka:
    print(f" {a}")
```

```
Data angka dalam list: [15, 30, 45, 60, 75]
Menampilkan satu per satu:
15
30
45
60
75
```

## ✓ Analisis:

Inisialisasi list statis berisi elemen numerik homogen. Proses traversal dilakukan menggunakan perulangan for langsung (for a in angka) untuk mengekstrak dan mencetak nilai tiap elemen.

```
# Tugas 2 - Simulasi stack sederhana
```

```
stack = []
```

```
# Push
```

```
stack.append(100)
stack.append(200)
stack.append(300)
print("Setelah push 100, 200, 300:", stack)
```

```
# Pop
```

```
nilai_pop = stack.pop()
print(f"Pop -> nilai yang diambil: {nilai_pop}")
print("Stack setelah pop:", stack)
```

```
# Peek (melihat elemen teratas tanpa menghapus)
```

```
if stack:
    print(f"Peek: {stack[-1]}")
```

```
Setelah push 100, 200, 300: [100, 200, 300]
Pop -> nilai yang diambil: 300
Stack setelah pop: [100, 200]
Peek: 200
```

## ✓ Analisis:

Selain menggunakan fungsi push (.append()) dan pop (.pop()), terdapat metode peek yang memanfaatkan indeks negatif stack[-1] untuk mengintip elemen teratas tanpa menghapusnya.

```
# Tugas 3 - Traversal data dengan perulangan

data = [5, 10, 15, 20, 25, 30]
print("Traversal data:")
for index, nilai in enumerate(data):
    print(f"  Index [{index}] -> Nilai: {nilai}")
```

```
Traversal data:
  Index [0] -> Nilai: 5
  Index [1] -> Nilai: 10
  Index [2] -> Nilai: 15
  Index [3] -> Nilai: 20
  Index [4] -> Nilai: 25
  Index [5] -> Nilai: 30
```

## Analisis:

Melakukan penelusuran list menggunakan fungsi bawaan enumerate(data). Fungsi tsb sangat efisien karena mampu mengembalikan dua nilai sekaligus dalam satu iterasi: indeks posisi dan nilai elemen itu sendiri.

## ✓ Level Sedang

```
# Tugas 4 - Menghitung jumlah iterasi dalam loop

ukuran_data = [5, 10, 20, 50]

for n in ukuran_data:
    counter = 0
    for i in range(n):
        counter += 1
    print(f"  n={n:3d} -> jumlah iterasi O(n) = {counter}")
```

```
n=  5 -> jumlah iterasi O(n) = 5
n= 10 -> jumlah iterasi O(n) = 10
n= 20 -> jumlah iterasi O(n) = 20
n= 50 -> jumlah iterasi O(n) = 50
```

## Analisis:

Berdasarkan output, jumlah iterasi yang dilakukan selalu sama dengan nilai  $n$ . Saat  $n = 5$ , loop berjalan 5 kali, dan ketika  $n = 50$ , loop berjalan 50 kali. Hal ini menunjukkan bahwa semakin besar ukuran data, semakin banyak iterasi yang dilakukan secara sebanding, sehingga algoritma memiliki kompleksitas waktu  $O(n)$  atau linear.

## ✓ Level Sulit

```
# Tugas 8 - Simulasi sistem antrian mahasiswa (Queue)

from collections import deque

class AntrianMahasiswa:
    def __init__(self):
        self.antrian = deque()

    def daftar(self, nama):
        self.antrian.append(nama)
        print(f" [{nama}] masuk antrian. Posisi ke-{len(self.antrian)}")

    def panggil(self):
        if self.antrian:
            dipanggil = self.antrian.popleft()
            print(f" Memanggil: [{dipanggil}] | "
                  f"Sisa antrian: {len(self.antrian)} orang")
            return dipanggil
        else:
            print(" Antrian kosong.")
            return None

    def lihat_antrian(self):
        if self.antrian:
            print(f" Antrian saat ini: {list(self.antrian)}")
        else:
            print(" Antrian kosong.")

antrian = AntrianMahasiswa()

# Mahasiswa mendaftar
print("Mahasiswa mendaftar:")
for nama in ["Reza", "Sinta", "Tono", "Ulfa", "Vino"]:
    antrian.daftar(nama)

print("\nAntrian sebelum dipanggil:")
antrian.lihat_antrian()
```



```
print("\nMemanggil mahasiswa satu per satu:")
for _ in range(3):
    antrian.panggil()

print("\nAntrian tersisa:")
antrian.lihat_antrian()
```

Mahasiswa mendaftar:

```
[Reza] masuk antrian. Posisi ke-1
[Sinta] masuk antrian. Posisi ke-2
[Tono] masuk antrian. Posisi ke-3
[Ulfa] masuk antrian. Posisi ke-4
[Vino] masuk antrian. Posisi ke-5
```

Antrian sebelum dipanggil:

```
Antrian saat ini: ['Reza', 'Sinta', 'Tono', 'Ulfa', 'Vino']
```

Memanggil mahasiswa satu per satu:

```
Memanggil: [Reza] | Sisa antrian: 4 orang
Memanggil: [Sinta] | Sisa antrian: 3 orang
Memanggil: [Tono] | Sisa antrian: 2 orang
```

Antrian tersisa:

```
Antrian saat ini: ['Ulfa', 'Vino']
```

## Analisis:

Program mengimplementasikan ADT Queue dengan prinsip First In First Out (FIFO) menggunakan modul `collections.deque`. Operasi `.append()` digunakan untuk mengantre di belakang (enqueue), `.popleft()` digunakan untuk memanggil sekaligus menghapus elemen dari depan antrean (dequeue).